

第 4 章 串 (String)

本章主题：串的各种基本运算及其实现

教学目的：了解数据结构的基本概念，理解常用术语

教学重点：● 掌握串的基本概念及其基本运算

● 掌握串的存储结构

主要内容：

1. 串的基本概念
2. 串的存储结构
3. 串的基本运算及其实现



4.1 串类型的定义

4.2 串的实现和表示



4.1 串的类型定义

4.1.1 串的定义

❖定义：串 (string) 是由零个或多个任意字符组成的字符序列，**是数据元素为单个字符的特殊线性表。**

记为： $s = \text{“ } a_1 a_2 \dots a_n \text{”}$ ($n \geq 0$)

为何要单独讨论“串”类型？

- 1) 字符串操作比其他数据类型更复杂（如拷贝、连接操作）
- 2) 程序设计中，处理对象很多都是串类型。

说明：

- 1) 其中 s 是串名，用双引号括起来的字符序列是串的值。
- 2) a_i ($1 \leq i \leq n$) 可以是字母、数字或其他字符； n 为串中字符的个数，称为串的长度， i 是序号。

几个术语

空串：长度为0的字符串；

空格串：由空格字符组成的字符串，长度 ≥ 1 。

子串：字符串中任意个连续的字符构成的序列；

母串：包含该子串的字符串；

两串相等：两个字符串的长度相等且各对应位置上的字符都相同。

字符的位置：从1开始

子串的位置：该子串第一个字符的位置



例 1： 现有以下 4 个字符串：

a = "BEI" b = "JING" c = "BEIJING" d = "BEIJING"

问：① 他们各自的长度？

a = 3 , b = 4 , c = 7 , d = 8

② a 是哪个串的子串？在主串中的位置是多少？

a 是 c 和 d 的子串，在 c 和 d 中的位置都是 1

③ 空串是哪个串的子串？ a 是不是自己的子串？

空串是任意串的子串；任意串 S 都是 S 本身的子串，除 S 本身外， S 的其他子串称为 S 的真子串。



注意:

- 空串和空白串的不同。
- 空串是任意串的子串
- 任意串是其自身的子串



4.1.2 串的基本运算

1. 求串的长度 **StrLength(s)** ;
2. 串赋值 **StrAssign(s1,s2);**
3. 两个串的连接 **StrConcat(s1,s2,s)** 或 **StrConcat(s1,s2)**
4. 求某串的子串 **SubStr(s,i,len)** ;
5. 串比较 **StrCmp(s1,s2)** ;
6. 子串定位 **StrIndex(s,t);**
7. 插入子串 **StrInsert(s,i,t)** ;
8. 删除子串 **StrDelete(s,i,len)** ;
9. 置换 **StrRep(s,t,r)** 。



定义下列几个变量：

```
char s1[20] = "dirtreeformat", s2[20] = "file.mem";
```

```
char s3[30], *p;
```

```
int result;
```

(1) 求串长 (length)

```
int strlen(char* s);    // 求串的长度
```

例如： printf("%d",strlen(s1)); 输出 13;

(2) 串复制 (copy)

```
char *strcpy(char* to, char* from);
```

该函数将串 from 复制到串 to 中，并且返回一个指向串 to 的开始处的指针。

例如： strcpy(s3,s1); //s3="dirtreeformat"

(3) 联接 (concatenation)

`char* strcat(char* to, char* from)`

该函数将串 `from` 复制到串 `to` 的末尾，并且返回一个指向串 `to` 的开始处的指针。

(4) 串比较 (compare)

`int strcmp(char *s1, char *s2);` // 比较 `s1` 和 `s2` 的大小，
// 当 `s1 < s2`、`s1 > s2` 和 `s1 = s2` 时，分别返回小于 0、大于 0 和等于 0 的值

【例】 `result = strcmp("baker", "Baker");` // `result > 0`
`result = strcmp("12", "12");` // `result = 0`
`result = strcmp("Joe", "joseph");` // `result < 0`



(5) 字符定位

```
char *strchr(char *s,char c);  
// 找 c 在字符串 s 中第一次出现的位置,  
// 若找到, 则返回该位置, 否则返回 NULL  
【例】 p=strchr(s2,'.');//p 指向 "file" 之后的位置  
        if(p) strcpy(p,".cpp");//s2="file.cpp"
```

注意:

- ①上述操作是最基本的, 其中后 4 个操作还有变种形式:
strncpy , **strncat** 和 **strchr** 。
- ②其它的串操作见 C 的 **<string.h>** 。在不同的高级语言中,
对串运算的种类及符号都不尽相同;
- ③其余的串操作一般可由这些基本操作组合而成;



例 1、求子串

求子串的过程即为复制字符序列的过程，将串 S 中的第 pos 个字符开始长度为 len 的字符复制到串 T 中。

思路：1. 判断参数的合法性；

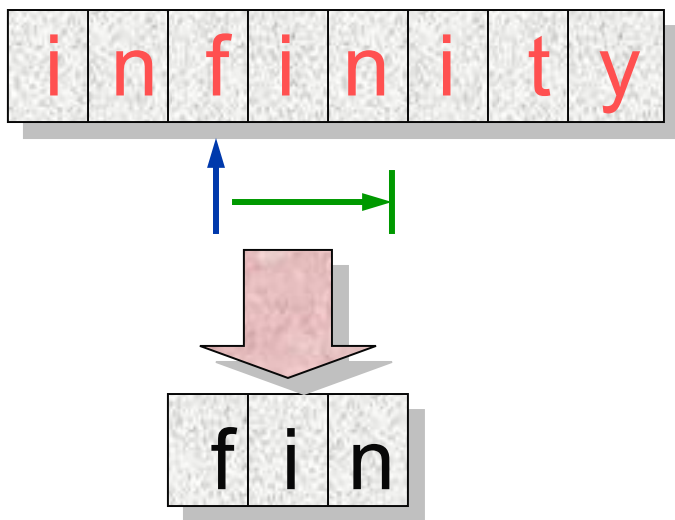
2. 调用 C 函数库接口，即基本串操作，完成复制；

```
void substr(char* sub,const char* s,int pos,int len)
{ /*s 和 sub 是字符数组，用 sub 返回串 s 的第 pos 个字符起长度为
  len 的子串其中  $0 \leq \text{pos} \leq \text{strlen}(s)-1$ , 且数组 sub 至少可容纳 len+1
  个字符。 */
  if(pos<0 || pos>strlen(s)-1 || len<0)
    error("parameter error")
  strncpy(sub,&s[pos],len); // 从 s[pos] 起复制至多 len 个字符到
  sub
}
```

例 2、串的定位 $\text{index}(s, t, \text{pos})$

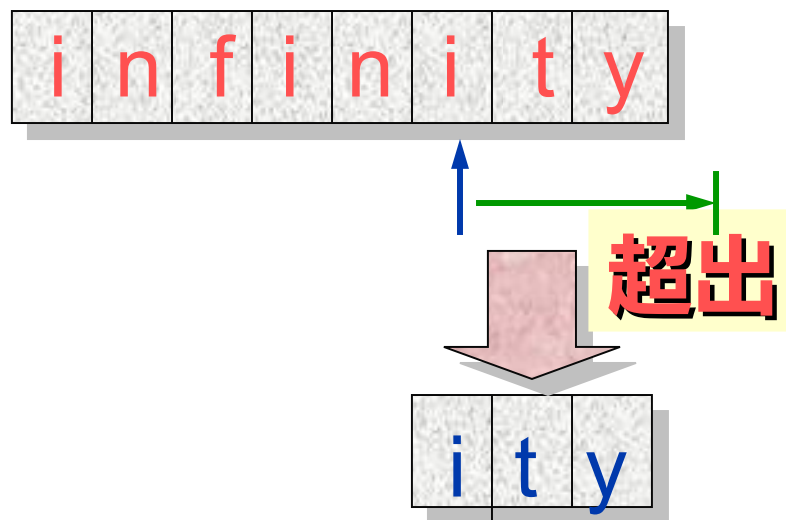
在主串 s 中取从第 pos 个字符起、长度和串 T 相等的子串和 T 比较，若相等，则求得函数值为 i ，否则值增 1 直至 S 中不存在和串 T 相等的子串为止。

$\text{pos} = 2, \text{len} = 3$



$\text{pos} + \text{len} \leq \text{curLen}$

$\text{pos} = 5, \text{len} = 4$



$\text{pos} + \text{len} > \text{curLen}$

例 2、串的定位 index(s,t,pos)

```
int index(string s,string t,int pos)
```

```
{  
    if(pos>0)  
    {  
        n=strlen(s);  m=strlen(t);  i=pos;  
        while(i<n-m+1) // 满足 pos+len<=n  
        {  
            substr(sub,s,i,m); // 从 s 中取从 pos 位置起 m 个字符的字符串给 s  
ub  
            if(strcmp(sub,t) !=0) // 比较 sub 和 t 是否相等  
                ++i; // 若不相等则继续比较  
            else  
                return(i);  
        }  
    }  
    return(0);  
}
```



4.2 串的实现

首先强调：串与线性表的运算有所不同，是以“**串的整体**”作为操作对象，例如查找某子串，在主串某位置上插入一个子串等。

串有三种机内表示方法：

- 顺序存储
 - **定长顺序存储表示**
——用一地址连续的存储空间存储值的字符序列，属**静态存储**方式。
 - **堆分配存储表示**
——用一地址连续的存储空间存储值的字符序列，但存储空间是在程序执行过程中**动态分配**而得。
- 链式存储
 - **串的块链存储表示**
——**链式**方式存储



4.2.1 定长顺序存储结构

定长顺序存储特点：用一组连续的存储单元来存放串，直接使用定长的字符数组来定义，数组的**上界预先给出**，故称为**静态存储**分配。

注：

- 一般用下标为 **0** 的数组分量来存放**串长**信息（如 pascal 语言）；
- C 语言约定在串尾加结束符 **'\0'**，以利操作加速，但不计入串长（**用首址和串长、或首址和尾标记来描述串数组**）
- 若字符串超过 Maxstrlen 则自动截断（**因为静态数组存不进去**）。



第一种:

```
#define MAXSIZE 256  
char s[MAXSIZE];
```

0	1	2	3	4	5	6	7	8	9	MAXSIZE-1
a	b	c	d	e	f	g	h	i	\0	_____

第二种:

```
#define MAXSIZE 256  
char s[MAXSIZE+1];
```

0	1	2	3	4	5	6	7	8	9	MAXSIZE
9	a	b	c	d	e	f	g	h	i	_____

运算实现（采用第一种表示串长的方式）

1. 求串的长度 S
trLength(s);

```
int StrLength(char s[])  
{int len=0;  
  while(s[len]!='\0')len++;  
  return len;  
}
```

2. 串赋值
StrAssign(s1,s2);

```
void StrAssign(s1,s2)  
char s1[ ], s2[ ];  
{int j=0;  
  while(s2[j]!='\0')  
    {s1[j]=s2[j];  
     j++;}  
  s1[j]='\0';  
}
```



- 3. 两个串的连接 StrConcat(s,s1,s2) ENTER
- 4. 求某串的子串 SubStr(s,i,len) ; ENTER
- 5. 串比较 StrCmp(s1,s2) ; ENTER



复习： C 语言中常用的串运算

注：用 C 处理字符串时，要调用标准库函数 `#include<string.h>`

C

类 C

串比较： `int strcmp(char *s1,char *s2);`

`StrCompare(S,T)`

求串长： `int strlen(char *s);`

`StrLength(S)`

串连接： `char* strcat(char *to,char *from)`

`Concat(&T, S1, S2)`

`Index(S, T, pos)`

子串定位： `char* strstr(char *s,char *c);`
注：Concat 操作（concatenation），
把多个短字符串合并为长字符串

.....



例 1 : 设 **s** = "I AM A STUDENT", **t** = "GOOD", **q** = "WORKER" 。求:

StrLength(**s**) = _ **14**

StrLength(**t**) = _ **4**

SubString(&**sub**, **s**, 8, 7) = "STUDENT"

SubString(&**sub**, **t**, 2, 1) = "O"

Index(**s**, "A", 0) = **3**

Index(**s**, **t**, 0) = **0**

Replace(&**s**, "STUDENT", **q**) = "I AM A WORKER"

Index(S, T, pos)
返回子串 T 在 pos
之后的位置

Replace(&S, T, V)
在 S 中用子串 V 替换子串

(s 中没有 t="GOOD")



提问： 当 $s = \text{'I AM A STUDENT'}$ 时，
 $\text{INDEX} (s, \text{'A'}, pos) = 3$ ，若想搜索后面那个
 'A' 怎么办？

答： $\text{INDEX} (s, \text{'A'})$ 返回的只是 **“第一次”** 出现的位置。

如果还要搜索后面的 A，则 pos 变量要跟着变才行。
也就是说，要把得到的 **“第一次”** 位置再代入
 $\text{INDEX} (s, \text{'A'}, pos)$ 函数中循环操作才行。



例 2： 设 **s** = “I AM A STUDENT”, **t** = “GOOD”, 求：

$\text{Concat}(\text{SubString}(\mathbf{s}, 6, 2), \text{Concat}(\mathbf{t}, \text{SubString}(\mathbf{s}, 7, 8))) = ?$

解： 因为

$\text{SubString}(\mathbf{s}, 6, 2) = \text{' ' A ' '}$;

$\text{SubString}(\mathbf{s}, 7, 8) = \text{' ' STUDENT' '}$

$\text{Concat}(\mathbf{t}, \text{SubString}(\mathbf{s}, 7, 8)) = \text{' ' GOOD STUDENT' '}$

所以：

$\text{Concat}(\text{SubString}(\mathbf{s}, 6, 2), \text{Concat}(\mathbf{t}, \text{SubString}(\mathbf{s}, 7, 8)))$

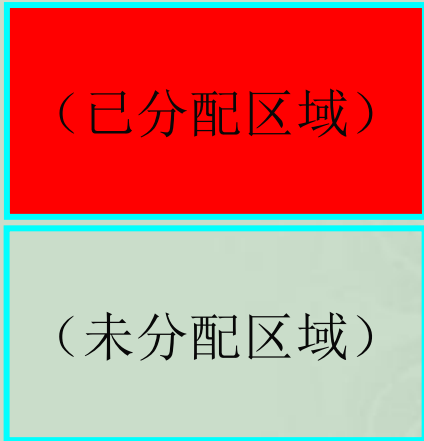
$= \text{' ' A GOOD STUDENT' '}$



4.2.2 堆存储结构

基本思想：

- ❖ 仍以一组空间足够大的、地址连续的存储单元存放串值字符序列，但它们的存储空间是在程序执行过程中动态分配的。所以也称为动态存储分配的顺序表。每当产生一个新串时，系统就从剩余空间的起始处为串值分配一个长度和串值长度相等的存储空间。



(已分配区域)

(未分配区域)

堆分配存储特点： 仍用一组连续的存储单元来存放串，
但存储空间是在程序执行过程中**动态分配**而得。

思路： 利用 malloc 函数合理预设串长空间。

特点： 若在操作中串值改变，还可以利用 realloc 函数
按新串长度增加空间 **(即动态数组概念)**

堆 T 的存储结构描述：

```
Typedef struct {  
    char *ch; // 若非空串，按串长分配空间； 否则  
    ch=NULL  
    int length; // 串长度  
} HString
```



例 1：编写建堆函数

```
Status StrAssign(HString &T, char *chars)
```

```
{ // 生成一个串 T, T 值 ← 串常量 chars
```

```
if (T.ch) free(T.ch); // 释放 T 原有空间
```

```
for (i=0, c=chars; c; ++i, ++c); // 求 chars 的串长度 i
```

```
if ( !i ) {T.ch = NULL; T.length = 0;}
```

```
else{
```

```
if (!(T.ch = (char*)malloc( i * sizeof(char))))
```

```
exit(OVERFLOW);
```

```
T.ch[0..i-1] = chars[0..i-1];
```

```
T.length = i;
```

```
}
```

```
Return OK;
```

```
} //StrAssign
```

C是指针变量，可以自增！意即每次后移一个数据单元。

直到终值为“假”
停止，串尾特征是
 $c = '\backslash 0' = \text{NULL}$

= 0

此处 T.ch[0] 没有用来装串长，因为另有 T.length 分量



例 2：用“堆”方式编写串插入函数

Status **StrInsert** (HString **&S**, int pos, **HString T**)

{ // 在串 S 的第 pos 个字符之前（包括尾部）插入串 T

if (pos<1||pos>S.length+1) return ERROR; //pos 不合法则告警

if(T.length){ // 只要串 T 不空，就需要重新分配 S 空间，以便插入 T

if (! (S.ch=(char*)**realloc**(S.ch, (S.length+T.length)*sizeof(char)))) exit(OVERFLOW); // 若开不了新空间，则退出

for (i=S.length-1; i>=**pos-1**; --i) S.ch[i+T.length] = S.ch[i];

// 为插入 T 而腾出 pos 之后的位置，即从 S 的 pos 位置起全部字符均后移

S.ch[**pos-1**...pos+T.length-2] = T.ch[**0**...T.length-**1**]; // 插入 T，略 \0

S.length += T.length; // 刷新 S 串长度

} return OK;

} //StrInsert



4.2.3 链式存储结构

链式存储特点：用链表存储串值，易插入和删除。

方法 1：链表结点的数据分量长度取 1（个字符）



方法 2：链表结点（数据域）大小取 n （例如 $n=4$ ）



讨论：法 1 存储密度为 $\frac{1}{2}$ ；方法 2 存储密度为 $\frac{9}{15} = \frac{3}{5}$

显然，若数据元素很多，用方法 2 存储更优—称为**块链结构**



块链类型定义:

对每个结点的描述

```
#define CHUNKSIZE 80 // 每块大小, 可由用户定义
typedef struct Chunk { // 首先定义结点类型
    char ch [ CHUNKSIZE ]; // 每个结点中的数据域
    struct Chunk * next; // 每个结点中的指针域
}Chunk;
```

```
typedef struct { // 其次定义用链式存储的串类型
    Chunk *head; // 头指针
    Chunk *tail; // 尾指针
    int curLen; // 结点个数
} LString; // 串类型只用一次, 前面可以不加 Lstring
```

对块链表的整体描述

块链函数的编写略



本章小结

串是一种特殊的线性表，其元素为字符。本章讨论了串的顺序存储结构、链式存储结构和堆分配存储结构。基于串的存储结构，可以实现串的各种基本运算，完成对串的处理。采用不同的存储结构，串的运算效率不同。

串的应用也比较广泛，我们将在课程设计中使用到串的模式匹配等其他算法。



```

Status StrConcat(SString &T, SString S1, SString S2) {
    // 用 T 返回由 S1 和 S2 联接而成的新串。若未截断，则返回 TRUE，
    否则 FALSE。
    int i;
    Status uncut;
    if (S1[0]+S2[0] <= MAXSTRLEN) { // 未截断
        for (i=1; i<=S1[0]; i++) T[i] = S1[i];
        for (i=1; i<=S2[0]; i++) T[i+S1[0]] = S2[i];
        T[0] = S1[0]+S2[0];
        uncut = TRUE;
    } else if (S1[0] < MAXSTRLEN) { // 截断 s2
        for (i=1; i<=S1[0]; i++) T[i] = S1[i];
        for (i=S1[0]+1; i<=MAXSTRLEN; i++) T[i] = S2[i-S1[0]];
        T[0] = MAXSTRLEN;
        uncut = FALSE;
    } else { // 截断 (仅取 S1)
        for (i=0; i<=MAXSTRLEN; i++) T[i] = S1[i];
        uncut = FALSE;
    }
    return uncut;
} // Concat

```



BACK

```
Status SubString(SString &Sub, SString S, int pos, int len) {  
    // 用 Sub 返回串 S 的第 pos 个字符起长度为 len 的子串。  
    // 其中,  $1 \leq \text{pos} \leq \text{StrLength}(S)$  且  $0 \leq \text{len} \leq \text{StrLength}(S) - \text{pos} + 1$  。  
    int i;  
    if (pos < 1 || pos > S[0] || len < 0 || len > S[0] - pos + 1)  
        return ERROR;  
    for(i=1; i<=len; i++)  
        Sub[i] = S[pos+i-1];  
    Sub[0] = len;  
    return OK;  
} // SubString
```



BACK


```
int StrComp(char *s1,char *s2)
{
    int i=0;
    // 判断某个串是否有结束，同时比较当前位置的字符是否相等
    while(s1[i]!='\0'&& s2[i]!='\0'&& s1[i]==s2[i])
        i++;
    return s1[i]-s2[i]; // 若不相等，则返回当前字符相减的值
}
```



BACK

课后作业：

1. 编写算法，求串 **S** 所含不同字符的总数和每种字符的个数。



BACK